

Antigravity Prompt Chain: AI-Assisted Online Shopping System

Five connected prompts for Antigravity (VS Code-style agentic IDE). Run them in order. Each prompt reads the previous prompt's output, builds its own deliverables, and writes exactly one documentation file recording Input, Process and Output.

0. One-time setup

Step 1. Create an empty folder `online-shopping-ai/`, open it in Antigravity, and start a new agent conversation (Planning mode, so it shows a plan before editing).

Step 2. Paste the block below once as the first message (or save it as a workspace rule, for example `.agent/rules/project.md`) so every later prompt inherits it.

PROJECT RULES (apply to every prompt in this chain)

Project: Online Shopping System for a retail company.

Modules: Product Catalog, Cart, Order Management, Payment, User Management.

Stack (change if needed): Java 17 + Spring Boot, React + Redux Toolkit, MySQL, RabbitMQ, Docker.

Folder layout (create as needed):

```
docs/           -> one documentation file per prompt (docs/01-...,
docs/02-...)
architecture/  -> architecture.json, schema, diagram, adr/
api/           -> OpenAPI YAML files
services/      -> backend code (product-service, cart-service, order-
service)
frontend/      -> React app
quality/        -> review and Sonar reports
prompts/        -> prompt library (every prompt used, plus refinements)
```

DOCUMENTATION RULE (mandatory, every prompt):

- Create ONLY ONE documentation file per prompt, at the path I give.
- Do not create any other .md summary or report file for that prompt, except files that are explicitly listed as deliverables.
- The documentation file must have exactly these top-level sections:
 1. INPUT - the prompt text used (verbatim), files read, parameters (temperature, max tokens, model), assumptions
 2. PROCESS - numbered steps actually performed, decisions made and why, validation runs, retries or re-prompts, errors and fixes
 3. OUTPUT - every file created or changed (path + one-line purpose),

results of checks, known gaps

4. HANDOFF - what the next prompt must read and rely on
- Document what really happened. Never invent results, scan numbers or coverage figures. If something could not be run, say so.

GENERAL BEHAVIOUR:

- Separate INTENT (the goal) from INSTRUCTIONS (the constraints) when you restate my request.
- Do not use APIs, libraries or endpoints that you cannot confirm exist. Flag uncertainty instead.
- Ask me before making any assumption that changes scope.

Reply "Rules loaded" and wait for Prompt 1.

Prompt 1: Requirements and Architecture

Covers: prompting for developers, LLM basics, AI in requirements and design, ADRs, when not to use AI. **Doc produced:** docs/01-requirements-architecture.md

PROMPT 1 - AI-ASSISTED REQUIREMENTS AND ARCHITECTURE

Role: You are a senior solution architect.

INTENT: A retail company wants a new Online Shopping System. I need business requirements converted into technical requirements and a justified architecture.

INSTRUCTIONS / CONSTRAINTS:

- Modules: Product Catalog, Cart, Order Management, Payment, User Management.
- Must support 10,000 concurrent users.
- REST APIs only, contract-first design.
- Compare Monolith vs Microservices and recommend one with reasons.
- Recommend a database model per service (tables or collections, keys, relationships) and say whether to share or separate databases.
- Suggest microservice boundaries and what each service owns.
- Payment: AI must NOT be used in payment processing logic. Explain why.

TASKS:

1. Write the requirement breakdown: business requirement -> functional requirement -> non-functional requirement. Give each an ID (BR-01, FR-01, NFR-01).
2. Produce architecture/architecture.json with fields: services, responsibilities, api_boundaries, data_ownership, events, risks (each risk with severity and mitigation).
3. Create architecture/architecture.schema.json (JSON Schema) and a

validation script `scripts/validate_architecture.py` (or `.js`). Run it. If validation fails, fix the JSON and re-run, and record each retry.

4. Produce a logical architecture diagram as `architecture/diagram.mmd` (Mermaid).
5. Write ADRs in `architecture/adr/` using the format Title, Status, Context, Decision, Consequences, Alternatives:
 - ADR-001: Why microservices (and the trade-offs)
 - ADR-002: Why no AI in payment processing logic
 - ADR-003: Database-per-service vs shared database
6. Create the requirement traceability matrix as a table: Requirement ID -> Service -> API -> (leave Test and UI columns as TBD for later prompts).
7. Explain tokens and context length: what a token is, how context limits affected how you worked here (for example, why you split the work into steps), and which temperature and max-token values you used or would recommend for this kind of task (low temperature for structured JSON, higher for brainstorming). If the IDE does not let you set these, say so honestly and state the recommended values.
8. Add a short "When NOT to use AI" section for this project.
9. Save the bad prompt ("Design an online shopping system.") and this good prompt side by side in `prompts/prompt-library.md` with a note on why the good one works better.

DOCUMENTATION: Follow the Documentation Rule. Write the single file `docs/01-requirements-architecture.md`. Put the requirement breakdown, traceability matrix, and the temperature/token explanation inside it. The architecture JSON, schema, diagram and ADRs are separate deliverable files.

Do not start coding services. Stop when done and list the files created.

Prompt 2: Backend API Scaffolding (Contract-First)

Covers: AI for backend, code generation patterns, structured outputs, validation and retries. **Reads:** `architecture/architecture.json`, ADRs, `docs/01-...` **Doc produced:** `docs/02-backend-api-scaffolding.md`

PROMPT 2 - BACKEND API SCAFFOLDING (CONTRACT-FIRST)

Role: You are a backend engineer.

INPUT TO READ FIRST: `architecture/architecture.json`, `architecture/adr/*`, `docs/01-requirements-architecture.md`. Use the service boundaries and data ownership decided there. If something conflicts, tell me instead of silently changing it.

INTENT: Product Service and Cart Service need REST APIs designed OpenAPI-

first.

INSTRUCTIONS / CONSTRAINTS:

- OpenAPI 3.0, output strictly in YAML, versioned under /v1.
- Product Service: GET /products, GET /products/{id}, POST /products.
- Cart Service: endpoints for get cart, add item, update quantity, remove item (use /v1/carts/...).
- Correct HTTP status codes (200, 201, 400, 404, 409, 500).
- One shared structured error model (code, message, details, timestamp, traceId) reused in both specs.
- Maintain backward compatibility: only additive changes inside v1. Describe the rule for breaking changes.

TASKS:

1. Generate api/product-service.openapi.yaml and api/cart-service.openapi.yaml.
2. Validate both with an OpenAPI linter (for example Spectral or openapi-generator validate). Record the validation log in quality/validation-logs/. If validation fails, RETRY by fixing the spec, and record every attempt.
3. Demonstrate retry vs re-prompt: deliberately run one case where a small schema error is fixed by retry, and one where the instruction itself was unclear and needs a re-prompt with a refined prompt. Store the refined prompts in prompts/prompt-library.md.
4. Generate Spring Boot controller scaffolds and DTOs for Product Service and Cart Service from the contracts (use openapi-generator or equivalent). Add a global exception handler that returns the shared error model, plus request validation (Bean Validation).
5. Add a validation layer that checks request/response against the contract (for example a contract test) and make the build run it.
6. Max token limit experiment: ask for the full Product spec with a very small output limit, record where it was truncated and what broke, then show the fix (split into smaller generations or a continuation prompt). If you cannot set a token limit, simulate it by requesting in chunks and report honestly that it was simulated.
7. Update the traceability matrix from Prompt 1: fill the API column for each requirement.

DOCUMENTATION: Follow the Documentation Rule. Write only docs/02-backend-api-scaffolding.md. Include the prompt refinement history, the retry vs re-prompt demonstration, and the token experiment inside this single file. Validation logs live in quality/validation-logs/ as deliverable files.

Stop when done and list the files created.

Prompt 3: Frontend Generation and UI Risk Control

Covers: AI for frontend, accessibility, UI pitfalls, state bugs, regression risk. **Reads:** both OpenAPI files, docs/02-... **Doc produced:** docs/03-frontend-ui-risk-control.md

PROMPT 3 - FRONTEND GENERATION, INTERACTIVITY AND UI RISK CONTROL

Role: You are a senior frontend engineer who cares about accessibility and polished, responsive interaction.

INPUT TO READ FIRST: api/product-service.openapi.yaml, api/cart-service.openapi.yaml, docs/02-backend-api-scaffolding.md. Use ONLY endpoints that exist in those contracts (for example /api/v1/products). Do not invent endpoints, fields or libraries. If an interaction needs something the API does not offer (for example server-side search, wishlist), either implement it client-side and label it "client-only", or list it as an API gap. Never fake a backend call.

INTENT: Build an interactive, app-like customer UI for the shopping system, not a static set of pages.

INSTRUCTIONS / CONSTRAINTS:

- React with Redux Toolkit (or Angular if I say so). Pages: Product listing, Cart, Checkout.
- WCAG 2.1 AA throughout: ARIA labels, keyboard navigation, visible focus, labels on every form field, errors announced to screen readers, sufficient colour contrast. Every interactive feature below must work with keyboard only and with a screen reader.
- Handle loading, empty and error states on every data call, with a Retry button on errors.
- No direct state mutation (Redux Toolkit slices, immutable updates).
- Warn about and handle race conditions: stale responses when filters change, double-clicking Add to Cart, double-submitting checkout. Use AbortController or request-id checks and disabled/pending states.
- Checkout: no card data in state or logs (payment is a separate, non-AI service).

INTERACTIVE FEATURES TO BUILD:

1. Product listing: live search with debounce (300 ms), category and price filters shown as removable chips, sort dropdown, grid/list view toggle, pagination or "load more", skeleton loaders while loading, and filter state kept in the URL query string so the page can be shared and the Back button works.
2. Quick view: a product opens in an accessible modal dialog (role=dialog, aria-modal, focus trap, Esc to close, focus returns to the trigger).
3. Add to cart: optimistic update with rollback on failure, a toast with

Undo, and an aria-live region announcing "Added to cart" or the error. Button shows pending state and cannot be double-clicked.

4. Mini-cart drawer: opens from the header cart icon with a live item-count badge, quantity stepper (+ / -), remove with Undo, and a running subtotal that updates instantly.

5. Cart page: inline quantity editing with min/max limits and clear messages, "save for later" and wishlist heart toggle (client-only, persisted in localStorage), and an empty-cart state with a call to action.

6. Checkout: multi-step form (Shipping, Review, Payment) with a progress indicator (aria-current="step"), inline validation on blur, error summary that receives focus on submit, a live order summary panel, and the ability to go back without losing data.

7. Motion and polish: smooth transitions that respect prefers-reduced-motion, a light/dark theme toggle that respects prefers-color-scheme, and keyboard shortcuts ("/" focuses search, "?" shows a shortcuts help dialog), all documented on screen.

8. Interactive component documentation: add Storybook (or equivalent) stories for key components in loading, empty, error and success states, so interactions can be demonstrated without the backend (use mock handlers such as MSW).

TASKS:

1. Generate frontend/ with the pages, components, API client, Redux slices and mock handlers above.

2. Run an accessibility audit (eslint-plugin-jsx-a11y plus axe-core in tests, or Lighthouse). Save the raw results to quality/accessibility/ and fix every failure you can. Record before and after. Also manually check keyboard-only flow and focus order for the modal, drawer and checkout steps.

3. Add end-to-end interaction tests (Playwright or Cypress) for: search + filter, add to cart with failure rollback, quantity change, checkout step navigation with validation errors. Save results to quality/e2e/.

4. State management improvement: list the state bugs you found or avoided (mutation, race conditions, stale data, optimistic update drift) and show the improved version.

5. Write the UI risk report: each risk (include focus traps, optimistic rollback, URL-state sync, localStorage drift, animation and motion sensitivity), likelihood, impact, mitigation.

6. Write the regression checklist (test points per page and per interaction).

7. Add the Prompt 3 library entry to prompts/prompt-library.md (prompt, purpose, known weaknesses, how to refine).

8. Update the traceability matrix: fill the UI column.

DOCUMENTATION: Follow the Documentation Rule. Write only docs/03-frontend-ui-risk-control.md. Include an interaction inventory (feature, component, keyboard behaviour, ARIA used, API endpoint or "client-only"), the UI risk

report, regression checklist and accessibility compliance summary (with evidence paths) inside it.

Stop when done and list the files created.

Prompt 4: Code Review and Quality Governance

Covers: AI code review mindset, SonarQube, complexity, hallucinated APIs, licensing risk.

Reads: everything built so far **Doc produced:** docs/04-code-review-quality-governance.md

PROMPT 4 - AI-ASSISTED CODE REVIEW AND QUALITY GOVERNANCE

Role: You are a strict code reviewer and quality gate owner.

INPUT TO READ FIRST: services/ (Product, Cart, and an Order Service skeleton if present), frontend/, docs/02 and docs/03. If there is no Order Service code yet, create a small Order Service (create order, get order) first, with realistic but deliberately imperfect code is NOT allowed: write it normally, then review it honestly.

INTENT: Before deployment, review the code for quality, security and licensing risk.

CHECKLIST:

- Missing null checks, code coverage gaps, cyclomatic complexity, readability
- Security flaws and hardcoded secrets
- Hallucinated or non-existent APIs/methods/dependencies
- Licensing concerns in dependencies (list each dependency and licence, flag copyleft or unknown)

TASKS:

1. AI review: produce quality/ai-review.json with exactly this shape:

```
{ "security_issues": [], "complexity_score": "", "refactor_suggestions": [], "risk_level": "" }
```

Each security issue needs file, line, description, severity. Validate the JSON against a schema; if it fails, retry and log it.

2. Run SonarQube: start SonarQube Community with Docker, run the scanner (Maven or sonar-scanner) on the backend, and export the results to quality/sonar-report.md. If the scan cannot run here, tell me the exact commands for me to run and mark the Sonar section as pending. Do not make up numbers.

3. Compare AI review vs Sonar: what each found, what only one found, false positives, and which tool is better for which kind of issue.

4. Fix the detected issues and refactor the worst-complexity code. Keep

behaviour the same and keep tests passing.

5. Re-run the scan or review and record the before/after.
6. Write a quality improvement summary.
7. Add this prompt to prompts/prompt-library.md.

DOCUMENTATION: Follow the Documentation Rule. Write only docs/04-code-review-quality-governance.md. Include the AI vs Sonar comparison, differences table, and quality improvement summary inside it. ai-review.json, sonar-report.md and the refactored code are the other deliverables.

Stop when done and list the files created.

Prompt 5: Testing and Edge Case Simulation

Covers: AI in testing, test generation, edge cases, output validation, retry vs re-prompt.

Reads: refactored code from Prompt 4, docs/04-... , traceability matrix **Doc produced:** docs/05-testing-edge-cases.md

PROMPT 5 - AI-GENERATED TESTING AND EDGE CASE SIMULATION

Role: You are a QA automation engineer.

INPUT TO READ FIRST: the refactored services, docs/04-code-review-quality-governance.md, the traceability matrix from docs/01. Test the refactored code, not the original.

INTENT: Cart and Payment modules need robust testing.

INSTRUCTIONS:

- JUnit 5 + Mockito. Payment is tested against a mocked/stubbed payment gateway client (no real payment calls, and no AI in payment logic).
- Generate test cases in a structured format: test name, scenario, expected result, code snippet.
- CartService coverage: normal flow, empty cart, large quantity, concurrent modification, invalid product ID.
- Boundary tests: quantity 0, 1, max allowed, max+1, negative, price rounding.
- Simulations: out-of-stock, payment timeout (with retry/idempotency behaviour), invalid coupon.
- Security scenario tests: injection in product ID or coupon code, access to another user's cart, tampered price in the request.

TASKS:

1. Generate the test suite under services/*/src/test/java.
2. Produce the structured test cases as quality/test-cases.json and validate

it against a schema. Show one run where the output fails validation and is fixed by RETRY, and one that needs a RE-PROMPT because the instruction was ambiguous. Log both.

3. Run the tests with JaCoCo and save the coverage report to quality/coverage/. Report the real numbers. If coverage is below 80% for Cart and Payment, add tests until it is, or explain why not.

4. Write the edge case document: each edge case, why it matters, test that covers it, result.

5. Finish the traceability matrix: fill the Test column so every requirement has a service, API, UI and test. List any requirement with no test.

6. Add the final prompt-library entry and refined versions of all five prompts.

DOCUMENTATION: Follow the Documentation Rule. Write only docs/05-testing-edge-cases.md. Include the edge case document, the security scenario results, and the completed traceability matrix inside it. The test code, test-cases.json and coverage report are the other deliverables.

Stop when done and give a final project summary: files per prompt and open risks.

How the chain connects

Prompt	Reads	Produces	Hands off
1	Project rules	architecture.json, ADRs, diagram, requirements, docs/01	Service boundaries, data ownership, traceability matrix
2	architecture.json, ADRs	OpenAPI specs, backend scaffold, validation logs, docs/02	API contracts the UI must follow
3	OpenAPI specs	React UI, a11y audit, risk report, docs/03	Working code to review
4	All code	AI review, Sonar report, refactored code, docs/04	Clean code to test
5	Refactored code, matrix	Tests, coverage, edge cases, docs/05	Final traceability and risk list

Final deliverables checklist

- **Use Case 1:** architecture.json, ADRs, requirement breakdown, temperature and token explanation (in docs/01)
- **Use Case 2:** OpenAPI files, backend scaffold, validation logs, prompt refinement notes (in docs/02)
- **Use Case 3:** UI code, accessibility proof, UI risk report, prompt library entry (in docs/03)
- **Use Case 4:** AI review report, Sonar report, refactored code, quality summary (in docs/04)
- **Use Case 5:** test code, coverage proof, edge case document, security scenario tests (in docs/05)

Tips for running in Antigravity

- Run one prompt per conversation turn and review the plan before approving edits.
- If the agent creates extra summary files, remind it of the one-doc rule and ask it to delete them.
- Commit to git after each prompt so you can roll back a bad run.
- Keep the screenshots or terminal output of validation, Sonar and coverage runs; they serve as proof for evaluation.